

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	<i>Facon, Janna Mae L.</i>
Section / Class	<i>BSIT- 3G</i>
Challenge Name	<i>MultiCode</i>
Category	<i>General Skills</i>
Points / Difficulty	<i>Easy</i>
Date Solved	<i>8/13/2026</i>
Flag Format Given	<i>PicoCTF{...}</i>

1. Challenge Description

Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copy-paste the original prompt.

The MultiCode challenge involved a suspiciously encoded message that contained a hidden flag. The challenge explained that there was no encryption, but instead multiple layers of obfuscation that needed to be removed one by one. The first step was to download the provided message.txt file and then identify and decode each layer until the original flag was revealed.

2. Initial Analysis / Reconnaissance

What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?

*The first step I did was read the challenge description carefully. I noticed that the challenge was categorized under **General Skills** and that the message contained multiple layers of encoding or obfuscation rather than encryption. I downloaded the provided message.txt file and used ls to confirm that the file was present in the terminal. I then used cat message.txt to examine its contents. The output appeared as a long string of characters, which indicated that the message was encoded rather than being plain text.*

*From the appearance of the data, I identified **Base64** as the first likely encoding layer. After decoding it, I received a hexadecimal-looking string. I then used xxd -r -p to convert the hexadecimal data back into text. The resulting text contained percent-encoded characters, indicating another layer using **URL encoding**. I decoded the URL encoding using*

Python's `urllib.parse.unquote()`, which produced another encoded string beginning with `cvpbPGS`. This pattern indicated **ROT13**, so I used `tr` to apply ROT13 and reveal the final flag.

3. Tools and Commands Used

List every tool/command you used, in the order you used them, and what each one told you.

Tool / Command	What it was used for	Result / Output
<code>wget</code>	Downloaded the provided message.txt file from the challenge file server.	The file was successfully downloaded as message.txt.2 because a file with the same name already existed.
<code>ls</code>	Checked the files in the current directory and confirmed that message.txt was available.	The directory contained message.txt along with the other challenge files.
<code>cat message.txt</code>	Displayed the contents of the suspicious message.	A long encoded string was displayed, showing that the message was not plain text.
<code>base64 -d</code>	Decoded the first Base64 encoding layer.	The output became a long hexadecimal string.
<code>xxd -r -p</code>	Converted the hexadecimal string back into its text representation.	The output contained percent-encoded characters, showing another encoding layer.
<code>Python urllib.parse.unquote()</code>	Decoded the URL/percent encoding.	The output became <code>cvpbPGS{arfgrq_rap0qvat_66b54257}</code> .
<code>tr 'A-Za-z' 'N-ZA-Mn-za-m'</code>	Applied ROT13 to the remaining encoded text.	The final flag was revealed as <code>picoCTF{nested_enc0ding_66b54257}</code> .

4. Solution Walkthrough

Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.

Step 1

The first step I did was read the challenge description. I learned that the message was not encrypted, but had multiple layers of obfuscation. I downloaded the provided message file using `wget` and then used `ls` to confirm that the file was available. I used `cat message.txt` to view the contents, and the result was a long string of unusual characters. This suggested that the message had been encoded and needed to be decoded layer by layer.

MultiCode

General Skills Easy

by Yahaya Meddy · picoCTF 2025

We intercepted a suspiciously encoded message, but it's clearly hiding a flag. No encryption, just multiple layers of obfuscation. Can you peel back the layers and reveal the truth?

Download the [message](#).

HINTS 1

1 [Reveal hint](#)

2 [Reveal hint](#)

FLAG 1

...

Submit

[Bookmark](#)

6,835

98%

[Open in Workspace](#)

Step 2

I first tried Base64 decoding because the contents had the appearance of a Base64-encoded string. I used: `cat message.txt | base64 -d` The result was a long sequence of hexadecimal characters. I then used `xxd -r -p` to convert the hexadecimal data back into readable text: `cat message.txt | base64 -d | xxd -r -p` This produced text containing percent-encoded values, which indicated that another layer of encoding was being used. I then used Python's `urllib.parse.unquote()` to decode the URL encoding: `cat message.txt | base64 -d | xxd -r -p | python -c "import sys, urllib.parse; print(urllib.parse.unquote(sys.stdin.read().strip()))"` The output was: `cvpbPGS{arfrgrq_rap0qvaf_66b54257}`

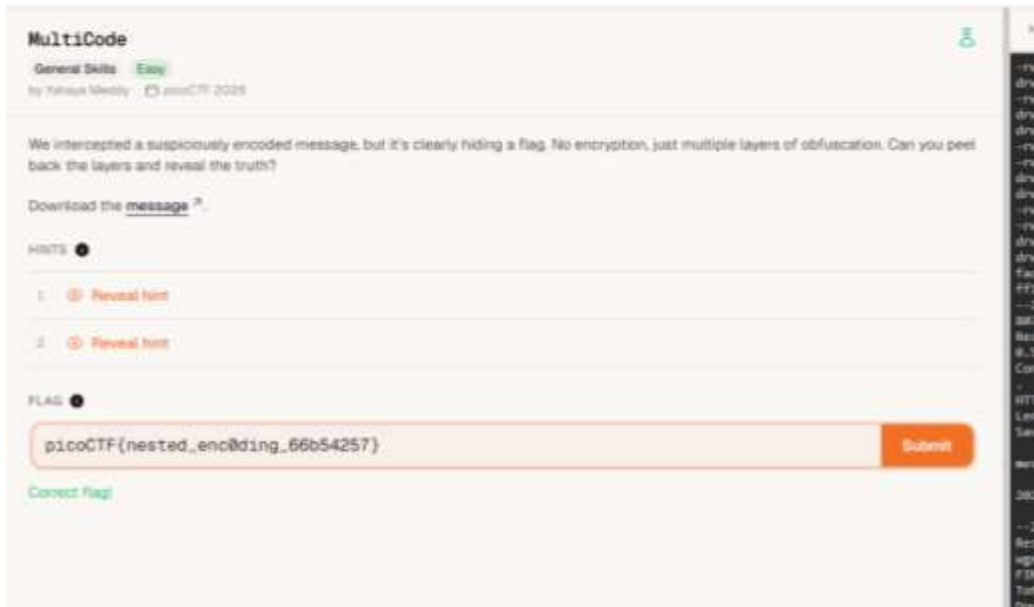
The screenshot shows the MultiCode challenge page on the left and a terminal window on the right. The terminal window displays the following commands and output:

```
cat message.txt | base64 -d | xxd -r -p | python -c "import sys, urllib.parse; print(urllib.parse.unquote(sys.stdin.read().strip()))"
```

The output of the command is: `cvpbPGS{arfrgrq_rap0qvaf_66b54257}`

Step 3

The result still did not look like a normal picoCTF flag, so I recognized that it was likely encoded using ROT13. I used the `tr` command to shift the alphabet by 13 positions: `cat message.txt | base64 -d | xxd -r -p | python -c "import sys, urllib.parse; print(urllib.parse.unquote(sys.stdin.read().strip()))" | tr 'A-Za-z' 'N-ZA-Mn-za-m'` This revealed the final flag: `picoCTF{nested_enc0ding_66b54257}` I then copied the flag and entered it into the FLAG field on the MultiCode challenge page.



5. Flag

`picoCTF{nested_enc0ding_66b54257}`

6. Key Concept / What I Learned

In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.

*This challenge taught me about **nested encoding and layered obfuscation**, where multiple encoding techniques are applied one after another to hide information. I learned how to recognize and reverse common encodings such as **Base64**, **hexadecimal**, **URL encoding**, and **ROT13**. I also learned how command-line tools can be combined into a pipeline to efficiently decode several layers and reveal the original information.*

7. References

List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.

<https://youtu.be/1MO5aHTnN1k?si=anPSVDrGjmAKrmsW>
